

National University of Sciences & Technology (NUST)



Instructor: Dr. Ayesha Maqbool
Parallel & Distributed Computing
CS13-E

Project Proposal
Distributed SGD: Parameter Server & Ring AllReduce

*An Adaptive Hybrid Framework for Communication-Efficient
Distributed Training*

Name	CMS
Muhammad Jahanzeb Babar	461484
Syed Fawwad Ahmed	463933
Muhammad Shaheer Saleh	467668

Table of Contents

1 Problem Statement.....	3
2 Distributed Systems Challenge.....	4
2.1 Communication Overhead.....	4
2.2 Scalability.....	4
2.3 Fault Tolerance.....	4
2.4 Consistency and Synchronization.....	5
2.5 Straggler Mitigation.....	5
3 Literature Review.....	6
Identified Limitations Across Literature.....	9
4 Gap Analysis.....	10
5 Proposed Hypothesis.....	11
H1.....	11
Sub-hypotheses.....	11
6 Performance Metrics.....	12
7 Experimental Plan.....	13
7.1 Implementation/Technology Stack.....	13
7.2 System Architecture Overview.....	13
7.3 Experimental Scenarios.....	14
7.4 Baseline Comparisons.....	14
9 References.....	15

1 Problem Statement

The rapid growth in deep learning model size, from millions to billions of parameters, has made single-machine training infeasible for many practical workloads. Distributed Stochastic Gradient Descent (SGD) has emerged as the dominant paradigm for parallelizing training across multiple workers. Two architectural patterns dominate the landscape:

- i. **Parameter Server (PS):** A centralized architecture where one or more servers maintain the global model. Workers compute local gradients and push them to the server, which aggregates and broadcasts the updated parameters [1], [2].
- ii. **Ring AllReduce (RAR):** A decentralized, peer-to-peer topology where each worker communicates only with its ring neighbors. Gradients are aggregated in a bandwidth-optimal manner without a central bottleneck [5], [14].

While each architecture has been studied independently, the following critical problem remains:

There is no unified experimental framework that (a) evaluates both architectures under identical, controlled conditions - including variable network latency, heterogeneous worker speeds, and node failures and (b) provides an adaptive mechanism that selects or blends the optimal communication strategy at runtime based on observed system conditions.

This project addresses that gap by designing, implementing, and evaluating a hybrid adaptive distributed SGD framework that dynamically switches between Parameter Server and Ring AllReduce modes to minimize total training time and maximize fault tolerance.

2 Distributed Systems Challenge

This section discusses how this project deals with some important problems that come up in parallel and distributed computing systems.

2.1 Communication Overhead

In distributed SGD, a lot of time is spent on sending and receiving gradients between workers. As we increase the number of workers, this communication becomes the main bottleneck.

- i. In the **Parameter Server (PS)** approach, all workers send data to a central server, which can become overloaded - a bottleneck.
- ii. In **Ring AllReduce (RAR)**, the communication is spread across workers, but the total delay increases as the number of workers in the ring grows.

2.2 Scalability

Scalability means how well the system performs when we increase the number of workers.

- i. **Amdahl's Law** says that speedup is limited if there is part of the system that cannot be parallelizable (like communication).
- ii. **Gustafson's Law** suggests better performance when the problem size increases along with the number of workers.

Hence, we use both models to quantify the parallel efficiency of our framework:

$$S_{\text{Amdahl}}(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

$$S_{\text{Gustafson}}(n) = n - \alpha \cdot (n - 1)$$

where p is the parallelizable fraction, n is the number of workers, and α is the serial fraction.

2.3 Fault Tolerance

This challenge of dealing with fault tolerance refers to how well the system handles failures. If a node (worker or server) fails during training, it can cause either loss of data, outdated gradients or a complete stopping of training.

- i. In **Parameter Server**, if the server fails, the whole system may stop (single point of failure).
- ii. In **Ring AllReduce**, if any one worker fails, the entire ring stops working.

2.4 Consistency and Synchronization

This refers to how workers stay updated with the latest model. Different methods include:

- i. **Bulk Synchronous Parallel (BSP)**, where all workers wait for each other before moving forward.
- ii. **Stale Synchronous Parallel (SSP)**, where workers can be slightly behind but within a limit.
- iii. **Asynchronous SGD**, where workers do not wait at all.

There is always a tradeoff when trying to procure either extreme; more synchronization does lead to a better accuracy but slower training whereas less synchronization leads to faster training but possible inconsistency.

2.5 Straggler Mitigation

In real systems, not all workers are equally fast. Some workers - called *stragglers* - are slower and delay the entire system. In synchronous systems, one slow worker can slow down everyone. Thus the system should be able to, firstly, detect slow workers and secondly, handle or reduce their impact. This is a very common real-world problem in distributed systems and directly affects performance.

A useful way to read this literature is to see it as a shift from early centralized training systems toward decentralized synchronization, followed by newer work that argues the best design depends on network conditions, scale, and system goals.

3 Literature Review

Distributed stochastic gradient descent has become central to large scale machine learning because modern datasets and models often exceed the speed and memory limits of a single machine. In distributed SGD, multiple workers process different mini batches of data in parallel and then synchronize their updates so that training still follows one shared model. The main systems challenge is that the benefit of using more machines can disappear if communication becomes too expensive. Across the literature, two broad synchronization ideas dominate this discussion: the parameter server, which relies on centralized coordination, and ring-allreduce or related decentralized methods, which spread communication across workers. The literature shows that the field did not move in a straight line from one clearly inferior method to one clearly superior method. Instead, it evolved toward a more conditional understanding of when each design works best [1], [2].

One of the earliest and most influential papers in this area is the parameter server framework proposed by Li et al. in 2013 [1]. This paper presents a client server architecture in which client nodes hold data and compute local updates, while server nodes maintain globally shared parameters. Its main contribution is broader than a simple communication pattern. The authors argue that distributed machine learning should be designed for realistic cloud environments, where machines may be unreliable, jobs can be preempted, and synchronous execution can slow badly because of stragglers. To address this, they emphasize asynchronous communication, flexible consistency models, elastic scalability, and fault tolerance. They also show that the framework is general enough to support several learning settings, including risk minimization, latent variable models, and deep learning. In this early stage of the literature, the parameter server is presented as a practical distributed systems solution that balances convergence needs with system efficiency [1].

This direction is developed further in the 2014 paper *Scaling Distributed Machine Learning with the Parameter Server* [2]. Here, the parameter server is framed as a mature systems platform for industrial scale machine learning. The paper identifies three major difficulties in large scale learning: huge bandwidth demand, synchronization barriers that hurt performance, and the growing need for fault tolerance in large deployments. In response, the authors describe five key features of their system: efficient communication, flexible consistency, elastic scalability, fault tolerance, and ease of use. An important point in this paper is that the system intentionally relaxes hard systems constraints when the underlying machine learning algorithm can tolerate mild inconsistency. This makes the parameter server literature important not only because it proposes centralized coordination, but because it introduces the broader idea that machine learning systems can trade exact synchronization for higher throughput [2].

A related extension of the centralized approach appears in *Gaia: Geo-Distributed Machine Learning Approaching LAN Speeds* [3]. Gaia studies a harder problem than ordinary cluster

training: machine learning across multiple data centers connected by wide area networks. The paper shows that simply running existing distributed machine learning systems across WANs can cause severe slowdowns, because WAN bandwidth is limited and moving raw data is often restricted by privacy or data sovereignty concerns. Gaia therefore keeps a parameter server style architecture but separates local synchronization inside each data center from cross data center synchronization. Its main systems contribution is Approximate Synchronous Parallel, which reduces insignificant WAN communication while still preserving correctness. This paper matters because it shows one of the main strengths of the parameter server approach: it can be adapted to difficult and bandwidth constrained environments rather than only fast local clusters [3].

The strongest challenge to centralized training in this set of papers comes from Lian et al. in 2017 [4]. Their decentralized PSGD paper asks directly whether decentralized algorithms can outperform centralized ones. The answer they provide is yes, under some conditions. Their argument is that centralized training places a heavy communication load on the busiest node, usually the parameter server, while decentralized methods distribute that load across neighbors in the network. The paper shows that decentralized PSGD can achieve similar computational complexity to centralized PSGD while reducing communication pressure on the hotspot node. The empirical results are especially important for this literature review because they show that on low bandwidth or high latency networks, decentralized methods can be up to ten times faster than well optimized centralized baselines. This paper shifts the debate by showing that decentralization is not merely a fallback choice when centralization is impossible, but can be a superior design choice in the right regime [4].

This shift became much more influential in practice with *Horovod: Fast and Easy Distributed Deep Learning in TensorFlow* [5]. Horovod argues that the standard parameter server setup in distributed TensorFlow suffered from both communication overhead and programming complexity. To solve this, the paper adopts ring allreduce as the main communication method. In ring allreduce, each worker exchanges gradient chunks with neighboring workers instead of sending everything to central servers. Horovod presents ring allreduce as attractive for two reasons. First, drawing on prior allreduce work, it treats the method as bandwidth optimal when buffers are large enough. Second, it significantly lowers the software burden, since users only need a few code changes instead of managing workers, servers, and cluster setup details. The paper reports that standard distributed TensorFlow on 128 GPUs left nearly half of the GPU resources underused, while Horovod improved scaling to about 88 percent efficiency and roughly doubled training speed on key benchmarks. For the literature, Horovod is important because it made ring allreduce both technically credible and practically easy to adopt [5].

A further step in the decentralized line is *Pipe-SGD* [6], which focuses less on the communication primitive alone and more on wall clock training time. The authors argue that earlier distributed training studies often paid too little attention to the real trade off between communication and computation. Their framework uses pipelined decentralized SGD to overlap these costs more effectively. On a four node GPU cluster, the paper reports wall clock improvements of up to 5.4 times over conventional centralized and decentralized baselines without loss of accuracy. The importance of this paper is that it broadens the question. After the rise of allreduce, the issue was no longer only whether centralized or decentralized

synchronization is better, but also how training systems can hide communication delay and reduce idle time during learning [6].

A broader comparative view is offered by *Performance Analysis and Comparison of Distributed Machine Learning Systems* [7]. This paper compares parameter server, peer to peer, and ring allreduce architectures for synchronous SGD using both analysis and experiments. Its key conclusion is that communication often dominates computation during distributed training. It also finds that parameter server throughput does not grow linearly as more servers are added because network congestion becomes a major issue, while ring allreduce achieves better performance through more efficient bandwidth use and by overlapping communication with computation. At the same time, the paper does not claim that one architecture is always best. It notes that parameter server designs may still make sense in weaker or less reliable environments, while Horovod style allreduce is more attractive when devices are fast and links are strong. This makes the paper useful because it turns the debate into a systems selection problem rather than a simple winner loser comparison [7].

A major turning point in the later literature is *Herring: Rethinking the Parameter Server at Scale for the Cloud* [8]. Herring argues that the field may have moved away from parameter servers partly because older implementations used bandwidth poorly, not because the architecture itself was fundamentally weak. The paper highlights several structural advantages of the parameter server. For a model of size M , each worker sends and receives M bytes and only needs two communication hops, whereas ring allreduce requires about $2M$ bytes and $2(n - 1)$ hops. Herring then identifies the practical reasons older parameter server systems scaled poorly, including unequal bandwidth allocation, bursty communication, and network congestion caused by poor sharding. To address these, it introduces balanced fusion buffers and uses AWS EFA to exploit cloud bandwidth more effectively. The results are striking: the paper reports faster gradient reduction than allreduce methods and BERTlarge training in 62 minutes on 2048 GPUs with 85 percent scaling efficiency. In the literature as a whole, Herring is important because it reopens the case for centralized methods and shows that the earlier success of ring allreduce may have reflected implementation quality as much as architectural superiority [8].

Recent work on ring allreduce has also become more nuanced. *Rina: Enhancing Ring-AllReduce with In-Network Aggregation in Distributed Model Training* [9] accepts that ring allreduce avoids the incast problem of the parameter server, but argues that it suffers from a long dependency chain in which delays at one worker can affect the entire synchronization process. The paper therefore combines ring allreduce within network aggregation through an agent worker model. Its goal is to improve throughput while allowing incremental deployment, which previous parameter server based in network aggregation designs handled poorly. Rina reports more than 50 percent higher throughput than a strong PS based in network aggregation baseline at the same hardware cost, and in some comparisons improves throughput by up to six times over traditional ring allreduce and parameter server baselines. This paper shows that decentralized methods are still evolving, especially through network aware hardware support [9].

The latest two sources in your set help place the whole debate in a wider frame. The 2025 survey by Provatas et al. [10] argues that parameter server research should be understood across five recurring themes: consistency control, network optimization, parameter management, straggler

handling, and fault tolerance. Its value for this review is that it confirms the parameter server remains a major research topic and that no narrow metric such as raw throughput alone can capture the full design space. The 2025 dissertation by Provatas [11] pushes this even further by studying hybrid approaches under the parameter server paradigm. It identifies stale gradients and consistency control as key challenges and proposes a strategy switch method that begins with synchronous communication and then moves to asynchronous updates, aiming to combine speed and stability. It also studies data distribution strategies to reduce variability across workers. Together, these two sources suggest that the newer literature is not treating the problem as a binary contest, but as a problem of adaptive control over synchronization, communication, and workload balance [10], [11].

This broader view is reinforced by *Birch SGD* [12], which proposes a unifying framework for analyzing and designing distributed SGD methods using weighted directed computation trees. Birch SGD treats synchronized SGD, local SGD, asynchronous SGD, and related methods as points in a larger design space rather than isolated algorithms. The paper argues that these methods share a common convergence structure but differ in trade-offs such as communication efficiency, update frequency, and bandwidth usage. Its experimental summary also shows that different methods become attractive under different communication conditions. This is a useful ending point for the literature because it captures the main lesson of the field. The real question is no longer whether parameter server is always better than ring allreduce or vice versa. The more important question is which synchronization design best matches the network, model size, hardware, and tolerance for staleness in a particular training setting [12].

Overall, the literature shows a clear evolution. Early work established the parameter server as a flexible and fault tolerant architecture for distributed machine learning [1], [2], [3]. Later work exposed the communication bottleneck of centralized systems and made decentralized training, especially ring allreduce, highly influential in practice [4], [5], [6], [7]. More recent work has challenged the idea that ring allreduce is automatically superior by showing that improved parameter server implementations, hybrid strategies, and even formal unifying frameworks all point toward a more conditional conclusion [8], [9], [10], [11], [12]. For a literature review on distributed SGD, the strongest takeaway is that the core research gap is no longer simply identifying which architecture is faster in general. The real gap is understanding how synchronization strategy should adapt to network conditions, scale, compression choices, and workload characteristics so that distributed training can remain both efficient and stable across different environments.

Identified Limitations Across Literature

- L1 No unified comparison:** Most papers evaluate only one architecture (PS or Ring) under architecture-specific benchmarks. Head-to-head comparisons under identical conditions are rare.
- L2 Homogeneity assumption:** Most experiments assume homogeneous workers and

network links, which is unrealistic in shared-cluster and cloud settings.

- L3 Static topology:** All reviewed systems use a fixed communication topology throughout training, regardless of runtime conditions.
- L4 Limited failure modeling:** Fault tolerance is either ignored or addressed with heavyweight mechanisms (replication, checkpointing) without evaluating graceful degradation.
- L5 Compression orthogonality:** Gradient compression techniques are evaluated independently of the communication topology, missing potential synergies.

4 Gap Analysis

Based on the literature review, we identify the following explicit gaps:

Dimension	Description
Unresolved Problem	No existing framework provides a runtime-adaptive mechanism that switches between Parameter Server and Ring AllReduce based on observed system metrics and conditions (bandwidth utilization, straggler lag, network partition, to name a few).
Insufficiently Addressed Trade-off	The trade-off between <i>centralized coordination overhead</i> (Parameter Server) and <i>ring latency sensitivity</i> (AllReduce) has not been characterized under heterogeneous and failure-prone conditions in a unified testbed.
Scalability Bottleneck	Parameter Server suffers from a fan-in bottleneck whereas Ring AllReduce suffers from single-node failure propagation. Neither alone is optimal across all cluster sizes and conditions.
Proposed Improvement	An Adaptive Hybrid Distributed SGD framework that monitors communication cost, straggler lag, and failure events at runtime and dynamically selects the optimal aggregation topology per training epoch or iteration.

5 Proposed Hypothesis

H_1 : *An adaptive hybrid framework that dynamically switches between Parameter Server and Ring AllReduce communication topologies based on runtime metrics (bandwidth saturation ratio, straggler detection threshold, and node liveness) will achieve lower total training time and higher fault tolerance than either static topology alone, across heterogeneous and failure-prone distributed environments.*

Sub-hypotheses:

- $H_{1.1}$:** Ring AllReduce will outperform Parameter Server in homogeneous, high-bandwidth settings with ≥ 4 workers.
- $H_{1.2}$:** Parameter Server will outperform Ring AllReduce under heterogeneous worker speeds (straggler scenarios) when combined with asynchronous updates.
- $H_{1.3}$:** The adaptive hybrid will match or exceed the better of the two static topologies in every tested scenario, with $\leq 5\%$ switching overhead.
- $H_{1.4}$:** Under single-node failure, the hybrid framework will recover and continue training within one synchronization round, while static Ring AllReduce will stall completely.

6 Performance Metrics

The following quantitative metrics will be measured and reported:

Metric	Unit	Description
Training Throughput	<i>samples/sec</i>	Number of data samples processed per second across all workers.
Speedup	$S(n) = T_1 / T_2$	Ratio of single-worker time T_1 to n-worker time T_2 .
Parallel Efficiency	%	$E(n) = S(n) / n \times 100$
Communication Overhead	<i>ms/iteration</i>	Time spent on gradient synchronization per training iteration.
Bandwidth Utilization	<i>GB/s</i>	Effective bandwidth consumed by gradient exchange.
Convergence Time	<i>seconds</i>	Wall-clock time to reach a target loss/accuracy threshold.
Straggler Sensitivity	% <i>degradation</i>	Throughput degradation when one worker is artificially slowed by 2×–5×.
Failure Recovery Time	<i>seconds</i>	Time from node crash detection to resumed training.
Switching Overhead	<i>ms</i>	Latency introduced by topology switching in the adaptive framework.

7 Experimental Plan

7.1 Implementation/Technology Stack

Component	Technology
Language	Python 3.10+
Parallelism	multiprocessing, threading, mpi4py (MPI for Python)
Communication	ZeroMQ (for PS), MPI collective ops (for Ring AllReduce)
Numerical Backend	NumPy / PyTorch (for gradient computation)
Monitoring	Custom metrics logger (CSV/JSON) + Matplotlib for visualization
Workload	Logistic Regression and a small CNN on MNIST/CIFAR-10

7.2 System Architecture Overview

The system consists of the following components:

1. **Worker Processes:** Each worker holds a local copy of the model, computes gradients on a data shard, and participates in synchronization via the active topology.
2. **Parameter Server (PS Mode):** A dedicated process aggregates gradients from all workers, updates the global model, and broadcasts updated parameters.
3. **Ring AllReduce (RAR Mode):** Workers form a logical ring. Gradients are reduced in $2(n - 1)$ steps using scatter-reduce and allgather phases.
4. **Adaptive Controller:** A lightweight monitor that observes per-iteration metrics (communication time, straggler lag, node heartbeats) and triggers topology switching when thresholds are breached.

7.3 Experimental Scenarios

Experiment	Variable	Purpose
E1: Scalability	Workers: 2, 4, 8, 16	Measure speedup and efficiency curves for PS, RAR, and Hybrid.
E2: Straggler	1 worker slowed $2\times$ – $5\times$	Compare throughput degradation across topologies.
E3: Node Failure	Kill 1 worker mid-training	Measure recovery time and accuracy impact.
E4: Bandwidth	Throttle network to 100 Mbps, 1 Gbps, 10 Gbps	Identify crossover point where RAR overtakes PS.
E5: Model Size	Small (LogReg) vs Medium (CNN)	Evaluate communication-to-computation ratio impact.
E6: Adaptive Switching	Mixed conditions	Validate hybrid switching logic and measure overhead.

7.4 Baseline Comparisons

- **Baseline 1:** Static Parameter Server (BSP).
- **Baseline 2:** Static Ring AllReduce.
- **Proposed:** Adaptive Hybrid (PS $\leftrightarrow$ RAR switching).

All three will be evaluated on every experiment (E1–E7) to ensure a fair and controlled comparison.

9 References

- [1] M. Li, L. Zhou, Z. Yang, A. Li, F. Xia, D. G. Andersen, and A. J. Smola, “Parameter Server for Distributed Machine Learning,” in *Proc. NeurIPS Workshop on Big Learning*, 2013.
- [2] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B. Y. Su, “Scaling Distributed Machine Learning with the Parameter Server,” in *Proc. 11th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2014, pp. 583-598.
- [3] K. Hsieh, A. Harlap, N. Vijaykumar, D. Konomis, G. R. Ganger, P. B. Gibbons, and O. Mutlu, “Gaia: Geo-Distributed Machine Learning Approaching LAN Speeds,” in *Proc. 14th USENIX Symp. Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 629-647.
- [4] X. Lian, C. Zhang, H. Zhang, C. J. Hsieh, W. Zhang, and J. Liu, “Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel SGD,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow,” *arXiv preprint arXiv:1802.05799*, 2018..
- [6] Y. Li, M. Yu, S. Li, S. Avestimehr, N. S. Kim, and A. Schwing, “Pipe-SGD: A Decentralized Pipelined SGD Framework for Distributed Deep Net Training,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [7] S. Alqahtani and M. Demirbas, “Performance Analysis and Comparison of Distributed Machine Learning Systems,” *arXiv preprint arXiv:1909.02061*, 2019.
- [8] I. Thangakrishnan, D. Cavdar, C. Karakus, P. Ghai, Y. Selivonchyk, and C. Pruce, “Herring: Rethinking the Parameter Server at Scale for the Cloud,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC20)*, 2020.
- [9] Z. Chen, X. Liu, M. Li, Y. Hu, H. Mei, H. Xing, H. Wang, W. Shi, S. Liu, and Y. Xu, “Rina: Enhancing Ring-AllReduce with In-Network Aggregation in Distributed Model Training,” *arXiv preprint arXiv:2407.19721*, 2024.

- [10] N. Provatas, I. Konstantinou, and N. Koziris, “A Survey on Parameter Server Architecture: Approaches for Optimizing Distributed Centralized Learning,” *IEEE Access*, vol. 13, pp. 30993-31015, 2025.
- [11] N. K. Provatas, *Study and Optimization of Distributed Deep Learning Under the Parameter Server Architecture*, Ph.D. dissertation, National Technical University of Athens, Athens, Greece, 2025.
- [12] A. Tyurin and D. Sivtsov, “Birch SGD: A Tree Graph Framework for Local and Asynchronous SGD Methods,” in *International Conference on Learning Representations (ICLR)*, 2026.